南京凌鸥创芯电子有限公司



BLDC 平台使用手册

版本：v1.0

@ 2024, 版权归凌鸥创芯所有

机密文件，未经许可不可扩散

第一章 绪论

凌鸥 BLDC 方案特点：

- ⋆ 基于磁饱和效应法的静态初始位置检测，启动无噪音，适合于凸极率较好的电机
- ⋆ 低速下 ADC 模式，高速 CMP 模式的双模式控制方式
- ⋆ 支持大转矩启动
- ⋆ 长时间堵转时不丢失转子位置信息
- ⋆ 超低速稳定运行
- ⋆ 硬件比较器联动 MCPWM 模块做过流保护，及时保护电路硬件
- ⋆ 集成度高，电路精简，节省 BOM 成本

第二章 BLDC 控制原理

对于三相直流无刷电机，若每个控制周期的导通两个绕组，在一个电周期内，有六种不同的控制信号控制功率管开通关断，使得定子电流有六种状态，即定子绕组的合成磁动势有六种状态。如果按照一定的顺序，对电机进行六次换相后，电机转子正好旋转一周。所以，方波控制又被称为六步换相控制。

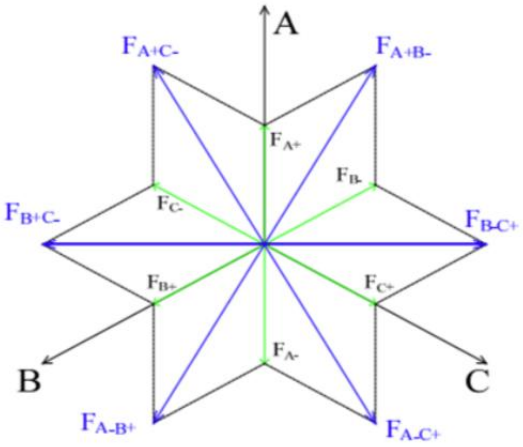


图 1.1　两两导通合成磁动势

2.1 最佳换相位置

方波控制时，需要不断进行换相操作，而在换相时转矩会发生突变。如果转矩波动过大，则电机会发生抖动，为了减少抖动现象，需要将力矩进行平滑变化，因此要选取最佳换相点。多种换相位置转矩波动分析：

（1）如果在转矩为 0 时进行换相，则每 60 度时的转矩变化图像如下：

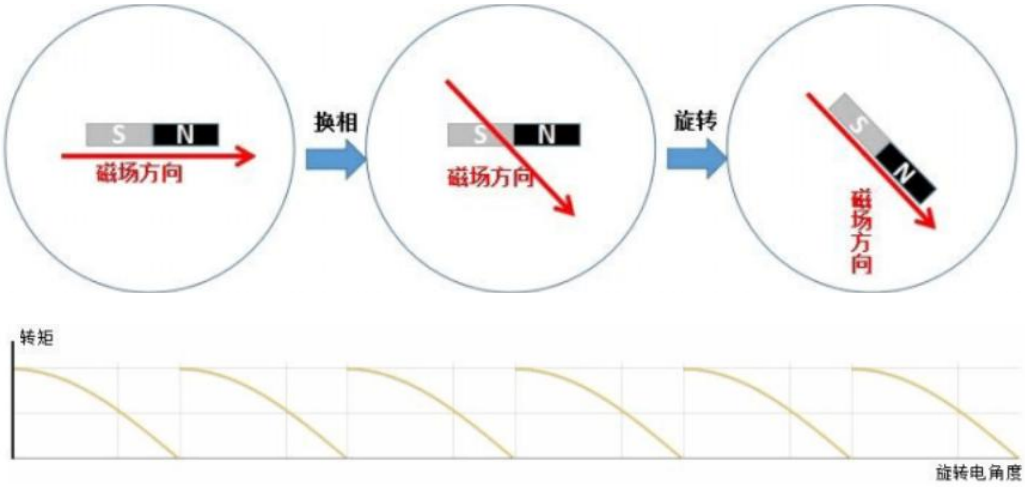


图 1.2　转矩为 0 时换相时转矩波动图

图 1.2 中，红色箭头为定子合成磁动势，转子磁动势由 S 极指向 N 极。

（2）如果在转矩为最大时（90 度）进行换相，则每 60 度时的转矩变化图像如下：

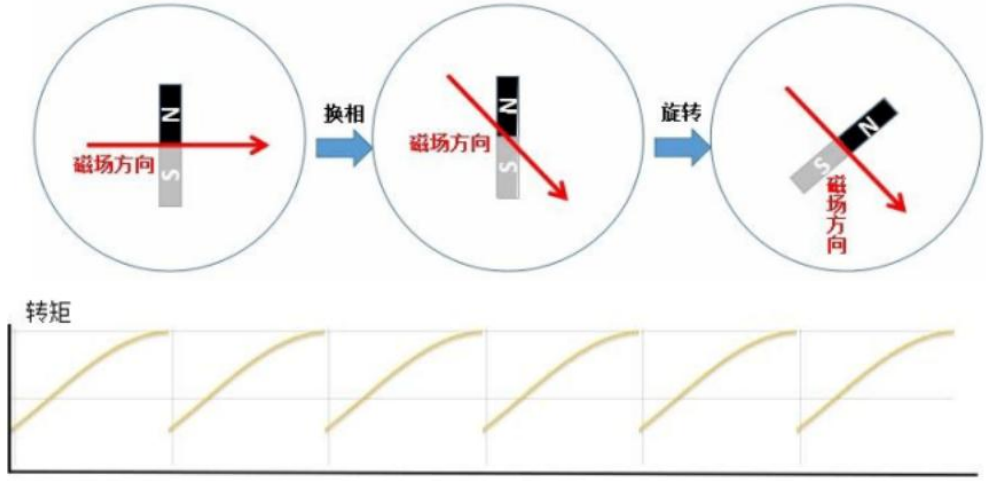
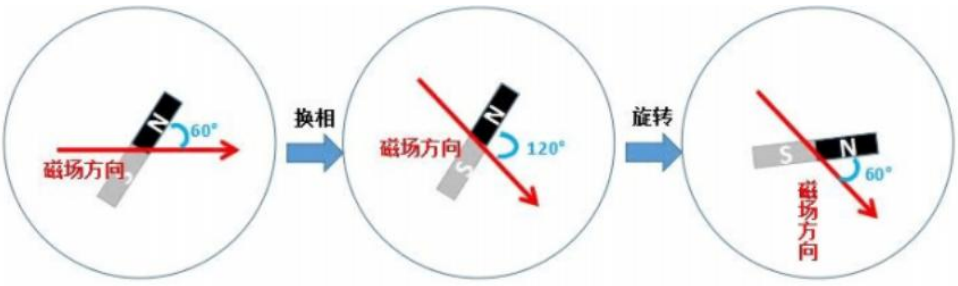


图 1.3　转矩最大时换相时转矩波动图

（3）如果在转矩为最大时落后 30 度时进行换相，则每 60 度时的转矩变化图像如下：



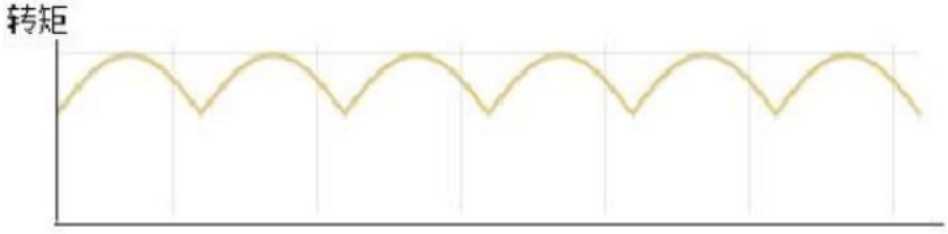


图 1.4　落后 30 度时换相时转矩波动图

通过上述图像对比可以明显看出，在转矩为最大时落后 30 度时进行换相，转矩脉动最平滑。

2.2 无感方波控制

通过上节分析，已经明确了定子磁场的最佳换相位置，但只有知道转子的实际位置，再结合理论才可以正确换相。

有感方波控制可以用霍尔传感器来确定转子位置，而无感方波控制一般采用反电动势来确定转子位置。

图 1.5 为三相逆变电路原理图，当 A 相上桥臂 TI 导通，B 相下桥臂 T6 导通，C 相悬空时，C 相就会有反电势的信息。因此可以检测悬空相反电势信息，然后延时 30 度进行换相。

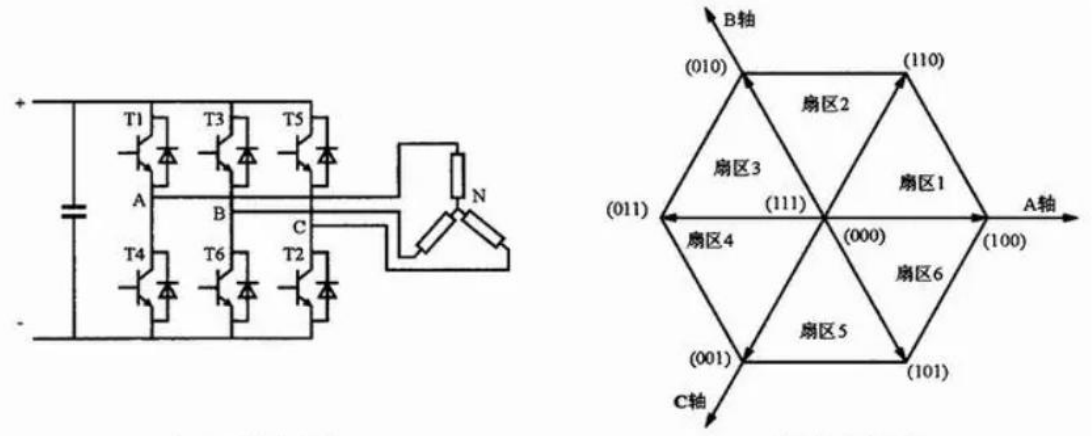


图 1.5　三相逆变电路及扇区原理图

电机转动时克服反电势做功，当悬空相反电势为 0 时，输出力矩达到最大，即达到过零点，只需要在经过 30 度延迟进行换相便可达到最佳控制效果。

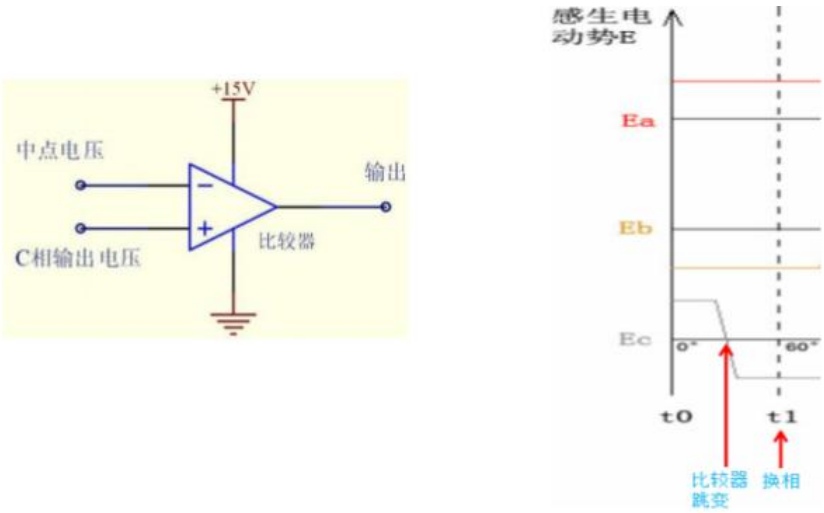


图 1.6　反电动势对应的换相点

当有中性点引出时，直接通过比较悬空相端电压与中性点的电压，其实也就是悬空相的反电势，便可进行过零判断。无中性点引出时，通过阻值相等的电阻重构出虚拟的中性点，来检测悬空相的反电势。

凌鸥 03x 系列 MCU，在内部已将中性点引出，只需要将三相反电动势信号 PA,PB,PC 连接到 CMPO_IP1,2,3 引脚上。

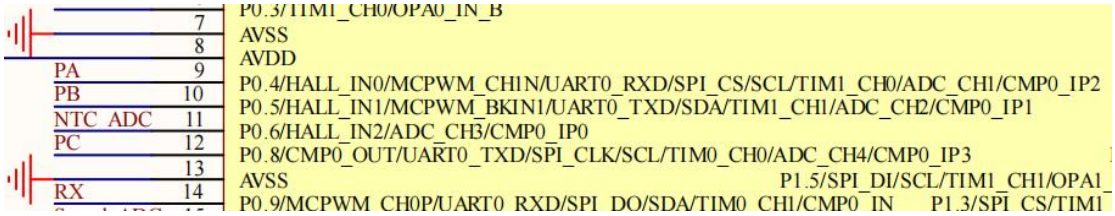


图 1.7　反电势引脚电路原理图

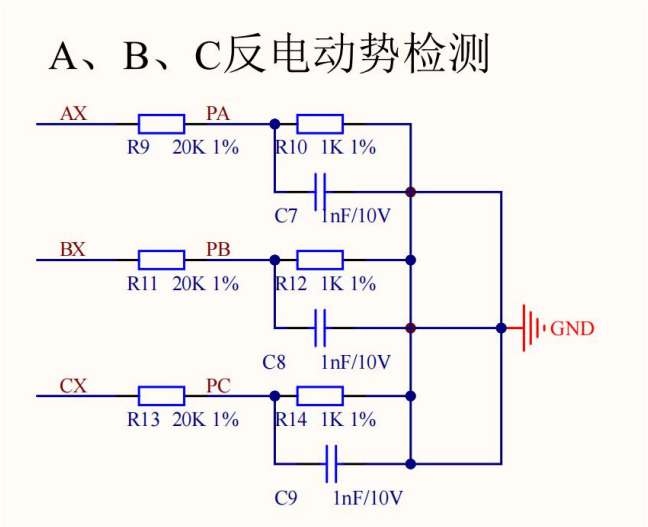


图 1.8　反电势检测电路原理图

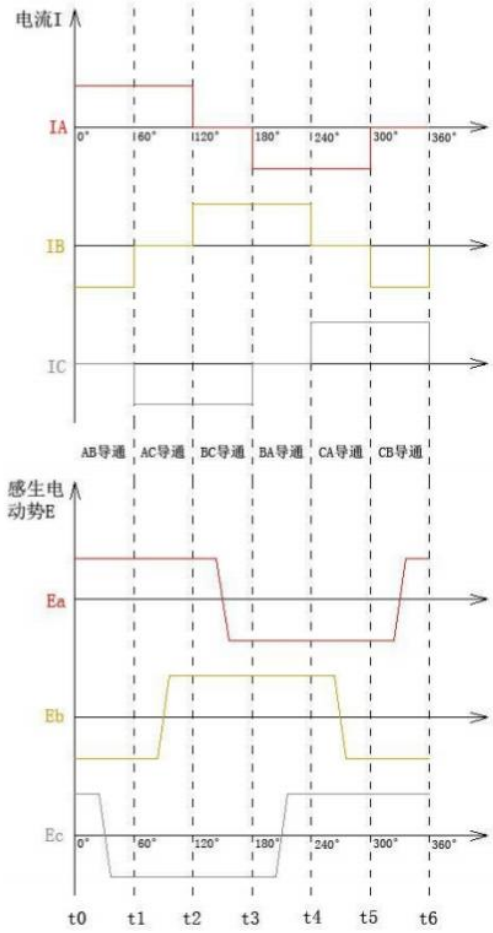


图 1.9　导通时所对应的换相点

CMP0_IP1/CMP0_IP2/ CMP0_IP3 三个端口引脚兼具 ADC 功能，可以同时做 ADC 模式和比较器模式，两种模式共用一个电路，只需配置软件即可完成两种模式的切换，可以有效节省外围电路。

ADC 模式：ADC 模式直接对悬空相电压进行采样判断，悬空相电压为母线电压一半时为反电动势过零点，然后再经过 30 度便可以换相。

CMP 模式：CMP 模式下将悬空相与中性点做比较，CMP 翻转点即为过零点，然后再经过 30 度便可以换相。

2.3 消磁(续流)处理

下图 1.10 中，红色箭头所指位置的波形就是消磁事件对应的波形。无论 PWM 在满或者不满占空比的时候，消磁事件都会发生；消磁事件发生的位置，就在悬空相线圈上；消磁事件发生的时刻，就在每次换相的的瞬间。因此，称之为换向消磁。

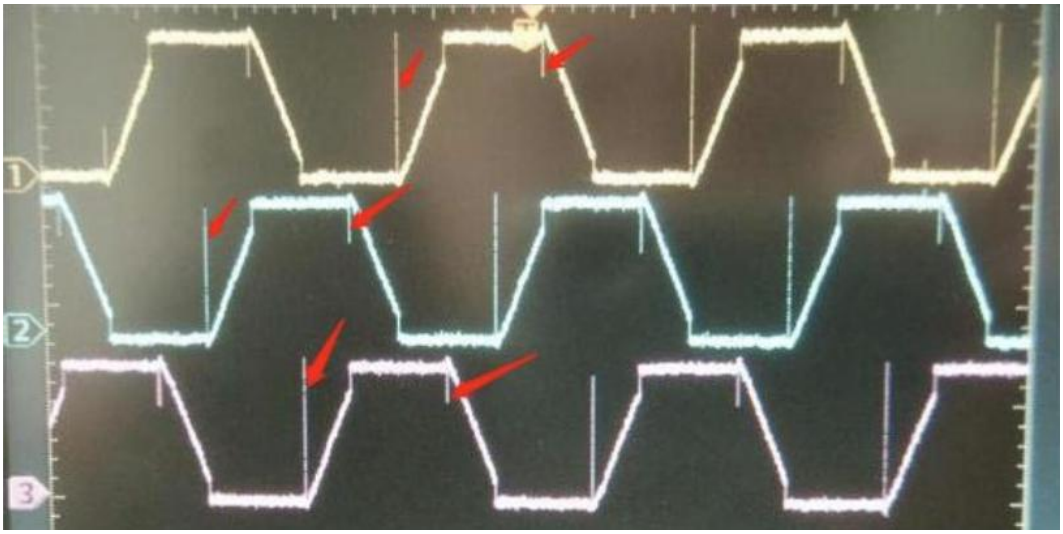


图 1.10　PWM 100%占空比时，UVW 相电压驱动波形

在换相的瞬间，悬空线圈会和续流二极管形成回路，完成续流。在续流过程中，会导致悬空线圈的电压要么近似等于 GND，要么近似等于电源电压 Vdc。续流事件的长短与电流大小有关，电流越大续流时间就越长；电流越小续流时间就越短，甚至有可能在示波器是看不到。

所以在比较器模式下会造成假过零事件，需要在软件中进行特别的处理，凌鸥 BLDC 平台在封装算法库中对续流事件进行处理，无需额外添加。

第三章 硬件参数配置

硬件初始化函数在“Hardware_init()”中, 更详细的硬件配置说明请参考 《LKS32MC03x User Manual》

注：03x 系列默认开门狗开启，调试时需要先进行关闭。

```
IWDG_DISABLE();
```

3.1 ADC 设置 & ADC 采样通道

本程序采用的 ADC 配置采用单段采样模式，量程为 3.6V, 数据左对齐，共有 8 个采样通道，需要根据硬件设计进行设置，在 hardware_config.h 当中。

```
ADC_InitStructure.Align = ADC_LEFT_ALIGN;                    // ADC 数据输出左对齐
ADC_InitStructure.Trigger_Mode = ADC_1SEG_TRG;               // 设置 ADC 转换模式为 1 段式采样
ADC_InitStructure.FirSeg_Ch = ADC_8_TIMES_SAMPLE;            // 第一段共采样 8 个通道
ADC_InitStructure.ADC_RANGE = ADC_RANGE_3V6;                 // ADC 量程选择
ADC_InitStructure.Trigger_En = ADC_HARDWARE_T0_TRG;          // 开启 T0 和 T1 硬件触发事件
ADC_InitStructure.IE = ADC_EOS0_IRQ_EN ;                     // 开启第一段中断

ADC_CHN0 = BEMF_CH_A | (BEMF_CH_B << 4) | (BEMF_CH_C << 8) | (ADC_PEAK_CUR_CHANNEL << 12);
ADC_CHN1=ADC_BUS_VOL_CHANNEL|(ADC_SPEED_CHANNEL<< 4) | (ADC_MOS_TEMP_CHANNEL << 8) | (ADC_BAT_TEMP_CHANNEL << 12);
```

3.2 GPIO 设置

P0.10、P0.11、P0.12、P0.13、P0.14、P0.15 是固定的 PWM 输出口，需要配置为输出模式，并复用为 MCPWM。

```
 /* MCPWM P0.10~P1.15 */
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_OUT; /*输出模式*/
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11 | GPIO_Pin_12 | GPIO_Pin_13 | GPIO_Pin_14 | GPIO_Pin_15;

GPIO_PinAFConfig(GPIO0, GPIO_PinSource_10, AF3_MCPWM);
GPIO_PinAFConfig(GPIO0, GPIO_PinSource_11, AF3_MCPWM);
GPIO_PinAFConfig(GPIO0, GPIO_PinSource_12, AF3_MCPWM);
GPIO_PinAFConfig(GPIO0, GPIO_PinSource_13, AF3_MCPWM);
GPIO_PinAFConfig(GPIO0, GPIO_PinSource_14, AF3_MCPWM);
GPIO_PinAFConfig(GPIO0, GPIO_PinSource_15, AF3_MCPWM);
```

Hold 引脚，初始化上电时置 1，扳机下电时置 0，达到延迟关断的效果，延迟时间可以自行设置。可以直接对以下函数进行调用，控制 Hold 引脚的输出。

```
#define LOCK_POWER_ON()                {GPIO0_PDO |= BIT1;}
#define LOCK_POWER_OFF()               {GPIO0_PDO &= ~BIT1;}
```

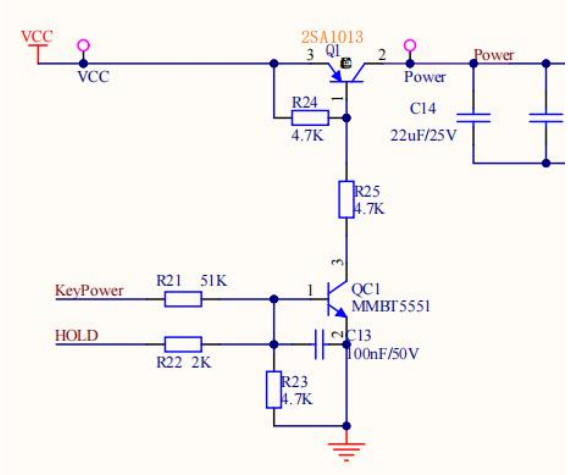


图 2.1 电源控制部分电路原理图

3.3 UTimer 设置

使用 Timer0 定时器，采用比较模式，计数值 TH 设置值为 24100，分频系数设置为 TIM_Clk_Div1（即 Timer 工作频率 48Mhz），则基数周期为 500us。为状态机函数 Task_Scheduler()提供时基，以便进行状态机更新，具体代码实现如下。

```
TIM_InitStruct.CLK_DIV = TIM_Clk_Div1;                  // 设置 Timer 模块数据分频系数
TIM_InitStruct.CH0_MODE = TIMER_OPMode_CMP;             // 设置 Timer CH0 为比较模式
TIM_InitStruct.CH0_POL = 0;                             // 计数器回零时，比较模式输出极性控制
TIM_InitStruct.TH = 24100;                              // 设置计数器计数模值 0.5ms 2KHZ
TIM_InitStruct.IE = TIM_IRQ_IE_ZC;                      // 开启 Timer 模块 CH1 比较中断
```

3.4 MCPWM 设置

MCPWM 模块采用中心对齐模式。

```
MCPWM_InitStructure.MCPWM_WorkModeCH0 = CENTRAL_PWM_MODE;
MCPWM_InitStructure.MCPWM_WorkModeCH1 = CENTRAL_PWM_MODE;   /* 通道工作模式设置，中心对齐或边沿对齐 */
MCPWM_InitStructure.MCPWM_WorkModeCH2 = CENTRAL_PWM_MODE
```

在硬件电路设计走线不便时，可通过交换通道 CHxP 和 CHxN 输出。

```
MCPWM_InitStructure.Switch_CH0N_CH0P = DISABLE;             /* 通道交换选择设置 */
MCPWM_InitStructure.Switch_CH1N_CH1P = DISABLE;
MCPWM_InitStructure.Switch_CH2N_CH2P = DISABLE;
```

通过与 CMP 模块联动，产生 FAIL 信号，关闭 PWM 输出。

```
MCPWM_InitStructure.FAIL1_INT_EN = ENABLE;                  /* FAIL1 事件 中断使能或关闭 */
MCPWM_InitStructure.FAIL1_INPUT_EN = ENABLE;                /* FAIL1 通道急停功能打开或关闭 */
MCPWM_InitStructure.FAIL1_Signal_Sel = FAIL_SEL_CMP;        /* FAIL1 事件信号选择，比较器*/
MCPWM_InitStructure.FAIL1_Polarity = HIGH_LEVEL_VALID;      /* FAIL1 事件极性选择 */
```

内置预驱型号的 MCU 需要设置：

```
MCPWM_SWAP = 0x67;
```

3.5 OPA 配置

运放增益设置 OPA_GAIN 为内部反馈电阻，R2:R1 的比值，本程序选择 200：10，外部电阻为 R0，最终的放大倍数为 R2/(R1+R0)。

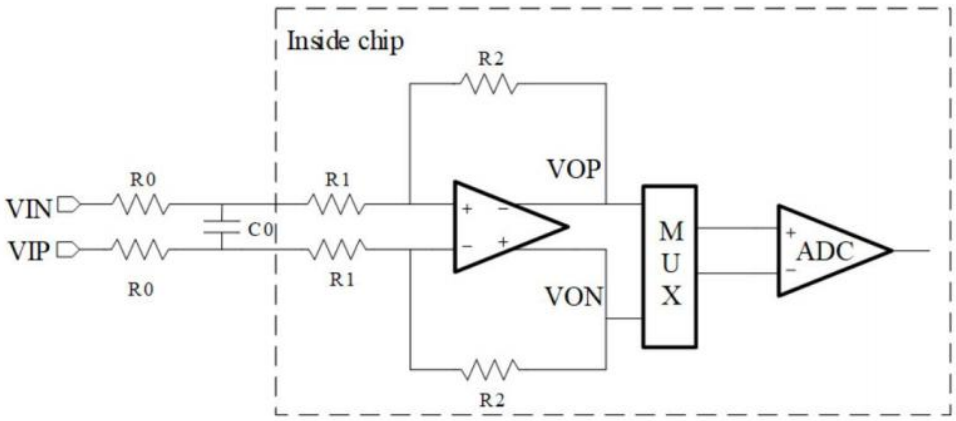


图 2.2　运算放大器结构图

```
OPA_InitStruct.OPA_Gain = PGA_GAIN_20;                    //OPA 内部增益电阻选择 200：10
OPA_InitStruct.OPA_B_EN = OPA0_IN_IP;                     //OPA0 使用 OPA0_IN_B/OPA0_IP_B 作为差分输入
```

3.6 DAC 配置

DAC 输出电压作为比较器的一端，作为过压基准。

```
mBLDC_CtrProc.OPA0_OUT_vol = Read_Trim(0x000001F4) & 0xFFFF;
/* OPA_OUT 共模电压读取,OPA0_OUT 取低 16 位，OPA1_OUT 取高 16 位，该数值是电压值放大 10000 倍
*/
short_value = (mBLDC_CtrProc.OPA0_OUT_vol + SHORT_CURRENT_VOL/2)*256/48000;
```

其中：

```
SHORT_CURRENT_VOL   =   (SHORT_BUS_CURRENT * RSHUNT * AMPLIFICATION_GAIN *10000)
```

式中，SHORT_BUS_CURRENT 为所设置的短路电流，RSHUNT 为采样电阻阻值，AMPLIFICATION_GAIN 为运放增益

3.7 CMP 配置

CMP0 的作用：CMP_SELP_IP1，2，3 交替作为正端，霍尔中性点作为负端，实现反电势过零点检测。

```
CMP_InitStruct.CMP0_SELP = CMP_SELP_IP1;                  // 比较器 0 信号正端选择
CMP_InitStruct.CMP0_SELN = CMP_SELN_HALL_MID;             // 比较器 0 信号负端选择
```

CMP1 的作用：OPA0_OUT 作为正端，DAC 作为负端，实现硬件过流保护。

```
CMP_InitStruct.CMP1_SELP = CMP_SELP_OPA0_OUT;    // 比较器正端选择，CMP1_SELP_CMP1_IP0
CMP_InitStruct.CMP1_SELN= CMP_SELN_DAC ;          // 比较器 1 信号负端选择
```

3.8 HALL 配置

03x 系列内部有两个定时器 Timer0 和 Timer1，为节省资源，利用 HALL 模块的计时器，通过超时溢出中断，进行计时功能。

```
HALL_InitStruct.OverFlow_IRQ_Ena = ENABLE;        /* 超时中断使能 */
HALL_InitStruct.CountTH = 9600000;                /* Hall 模块计数模值，计数超过模值会产生超时中断 */
```

第四章 软件说明

4.1 系统初始化程序

详细的系统初始化程序请参见函数“sys_init( )”

4.1.1 偏置采样以及电压保护

```
CurrentOffsetCalibration();      // 电流偏置读取
DC_offset_protect();             //母线电流偏置保护
```

4.1.2 BLDC 初始化

BLDC_Init()中，恒流控制时，当电流超过设定值降占空比：

```
BLDC_SensorlessCtr.BusCur_lim_cof = FRAC16(0.25);
```

低速标志位（ADC 模式），为 1 时则为低速状态，为 0 时为高速状态（CMP 模式），可在上位机中对此变量进行监测。

```
BLDC_SensorlessCtr.lowSpeedFlg = 1;
```

ADC 模式在采样电压为母线电压一半时，为过零点，但不同电机存在差异，可以通过此变量进行矫正。

```
BLDC_SensorlessCtr.halfVol_cof = FRAC16(0.52);        /* 反电势电压中点校正值 */
```

4.2 状态机函数

状态机按照电机的运行顺序来调度，处理函数为 Sys_State_Machine()，处理时间周期为 1ms。控制流程图如图 3-1 所示，状态机启动变量为“mBLDC_CtrProc.bMC_RunFlg”，当“struFOC_CtrProc.bMC_RunFlg ！= 0”时，状态机启动。

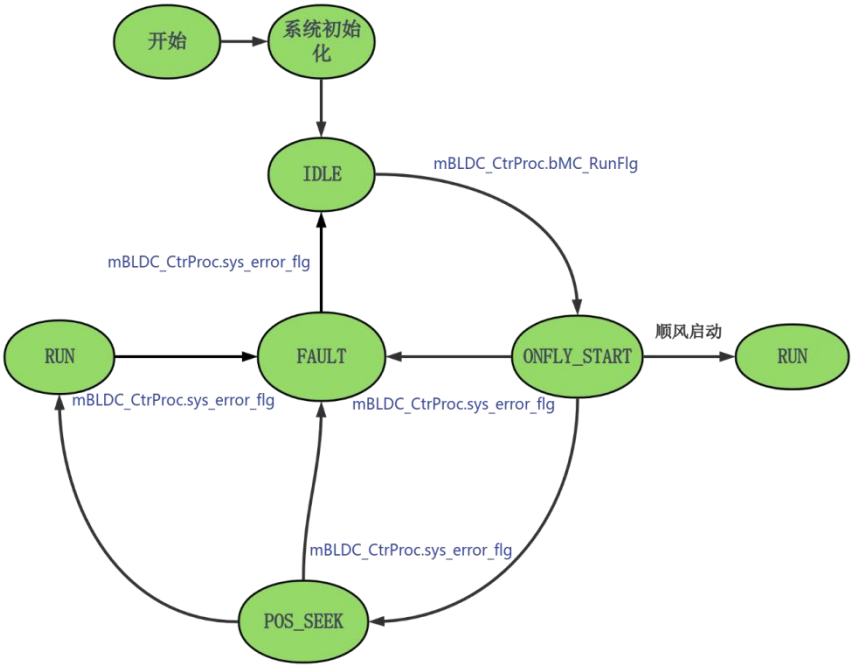


图 2.3　状态机调度流程图

常用的流程为：初始化-->初始位置检测-->闭环运行-->刹车状态-->空闲状态，故障检测监视每个状态的运行情况，一旦有故障发生，即“mBLDC_CtrProc.sys_error_flg!= 0”则进入故障状态，当故障解除后，即“mBLDC_CtrProc.sys_error_flg= 0”时，回到 IDLE 状态。具体各个状态的功能实现，这里面不再做详细介绍，请参考各个状态的程序。

4.2.1 换相函数

当初始位置检测完成后，通过 BLDC_Communication() 函数便可直接进行换相操作。因为逻辑相同，选取 case1 进行分析：

```
switch (BLDC_SensorlessCtr.communicateStep) {
case 1: {
        // A 相悬空，B 相 PWM C 相 ON ；检测下降沿过零事件 // A float | B PWM | C ON
 if((BLDC_SensorlessCtr.motorDirect == 0) && (BLDC_SensorlessCtr.onTheFlyFlg == 0)){
 //判断电机运行方向，进行重叠换相，为减少 DS 之间的电压
            MCPWM_UPDATE = 0;
            MCPWM_TH20 = 0;
            MCPWM_TH21 = 0;
            MCPWM_UPDATE = 0xff;
            MCPWM_IO01 = DRIVER_POLARITY | 0x1c;
            MCPWM_IO23 = DRIVER_POLARITY | 0x1c;
```

```
            while(HALL_CNT < RE_COMM_CNT) {;}
        }
        MCPWM_IO01 = DRIVER_POLARITY | (0x00c);
        if((BLDC_SensorlessCtr.lowSpeedFlg) && (BLDC_SensorlessCtr.onTheFlyFlg == 0)){
//判断是否为低速状态
            MCPWM_IO23 = DRIVER_POLARITY;
        }
        else{
            MCPWM_IO23 = DRIVER_POLARITY | 0x1c;
        }
SYS_AFE_REG1 = (CMP0_SELN_SET << 2) | (CMP1_SELN_SET << 4) | (CMP_IP_A << 8) |
(CMP1_SELP_SET << 12);      //比较器正负端配置，比较器正端选择悬空项
```

注：MCPWM_IO01 为 MCPWM CH0 CH1 IO 控制寄存器；MCPWM_IO23 为 MCPWM CH2 CH3 IO 控制寄存器，具体对应寄存器配置可以参考“LKS32AT03x_UM”用户手册。

在方波控制中，MOS 管从高电平切换到低电平时，电流会突然中断，这可能导致电流和电压的突变，从而产生 EMI 干扰和开关损耗。所以可以采用重叠换相，在方波信号的切换过程中引入一段时间的重叠，使得电流在切换时逐渐减小，然后再逐渐增加，从而减少了电流和电压的突变。在方波信号的切换时刻之前，提前降低功率半导体器件的导通程度，使得电流逐渐减小。

重叠换相时间 RE_COMM_CNT，通过在“MC_Parameter.h”中设置。

Case1 中 A 相悬空，B 相上桥导通，C 相下桥导通，从 B 相流入 C 相，但低速状态下“BLDC_SensorlessCtr.lowSpeedFlg”，C 相不能一直下桥导通，需要插入脉冲，保证低速度下稳定运行。

注：ADC 通道的配置在换相过程需要动态变换，已在库函数内部实现，所以用户不要修改 ADC_CHN0 的相关通道配置和采样顺序。

```
ADC_CHN0 = BEMF_CH_A | (BEMF_CH_B << 4) | (BEMF_CH_C << 8) |
(ADC_PEAK_CUR_CHANNEL << 12);
```

在进行完上述程序后，根据上升沿，下降沿，对比较器极性进行极性选择，程序如下：

```
if(BLDC_SensorlessCtr.BEMF_edgeFlg){
    CMP_CFG |= BIT0;
  }
   else{
    CMP_CFG &= (~BIT0);
  }
```

再判断是否处于 ADC 模式，如果为否，则说明处于比较器模式下，这时还需要根据滴答定时器进行消隐的处理。

```
if(mBLDC_CtrProc.nWork_mode == BLDC_MODE_ADC ){
        BLDC_SensorlessCtr.state = 0;
        BLDC_SensorlessCtr.blankEndTimeValue = 0;
    }
    else{
        CMP_delay = BLDC_SensorlessCtr.SpeedTimerCnt >> 1;
        SYSTICK_flg = 1;
        SysTick->CTRL = 0x0;
        SysTick->VAL = 0;
        SysTick->LOAD = 0;
        SysTick->LOAD = CMP_delay;
        SysTick->VAL = 0;
        SysTick->CTRL = 0x07;
    }
```

4.3 保护函数

4.3.1 电压保护函数

```
Vol_protect();              /* 电压保护 */
```

所对应的电压保护值，均通过在“MC_Parameter.h”中设置。

（1）欠压保护：

第一段欠压为运行中，第二段为启动前，有效对电池进行保护。

```
#define LOW_VOLATAGE_THD_1                  5       /* 第一段欠压 */
#define LOW_VOLATAGE_THD_2                  5.5     /* 欠压电压 */
```

（2）过压保护：

```
#define OV_VOLTAGE_THD                      25     /* 过压门槛 */
#define RSM_OV_VOLTAGE_THD                  25.5     /* 过压恢复门槛 */
```

4.3.2 分极限流函数

```
OVER_current_protect();
```

电动工具中有时需求，第一级限流只能运行 1000ms，第二级限流只能运行 200ms，在“MC_Parameter.h”中设置。

```
#define BUS_CURRENT_FIRST                    (u16)30   /* 一级限流 */
#define BUS_CURRENT_SECOND                   (u16)50   /* 二级限流 */

#define TIME_LIMIT_FIRST                   1000      /* 一级限流时间 */
#define TIME_LIMIT_SECOND                   200      /* 二级限流时间 */
```

4.3.3 堵转保护函数

Motor_Block_Protect();

当电机运行时，换相标志长时间不发生改变，即 BLDC_SensorlessCtr.communicateStep == BLDC_SensorlessCtr.oldCommunicateStep 时。

```
if(mBLDC_CtrProc.nBLDC_PWM_value > PWM_PERIOD * 0.25)
 // 当占空比大于 25%占空比时{
      block_THD = 100;          //堵转时间为 100ms
  }  //小于 25%占空比时
else{
      block_THD = 2000;     //堵转时间为 2000ms
  }
```

4.3.4 温度保护函数

TEMP_protect();

当采样 MOS 温度超过阈值时进行保护，在“MC_Parameter.h”中设置。

```
#define MOS_TEMP_UP_VOL                    5       //MOS 温度检测上拉电压，单位：V
#define MOS_TEMP_UP_RES                   10      //MOS 温度检测上拉电阻，单位：KΩ

#define MOS_TEMP_OVER_TIME                 500     //温度检测时间

#define MOS_TEMP_OVER_RES                 1.11   //MOS 过温时 NTC 阻值，80℃对应 1.67K   6512
#define RSM_MOS_TEMP_OVER_RES              1.2   //MOS 过温恢复 NTC 阻值，75℃对应 1.98K   7520
```

4.3.5 故障清除函数

Clear_error();

运行标志位为 0 时，对部分故障进行清 0。

```
if((mBLDC_CtrProc.Power_on_flg == 0) && (mBLDC_CtrProc.nPower == 0))
 {
```

```
        mBLDC_CtrProc.sys_error_flg &= ~(SHORT_ERROR | BLOCK_ERROR | HIG_VOL_ERROR |
        LOW_VOL_ERROR | OVER_LOAD_ERROR | PHASE_DROP_ERROR);
}
```

4.4 功能函数

4.4.1 扳机信号输入函数

```
Speed_control();            /*扳机信号输入处理*/
```

将输入信号转换成对应的占空比进行输出。

分级启动，超过 460 进行无级调速：

```
if(Speed_value < 460){
            Powr_value = START_POWER;
      }
      else{
            Speed_value = Speed_value - 460;
            Powr_value = Speed_value * 19;;      // 放大系数
            Powr_value += (s32)((Speed_value * FRAC16(0.06))) >> 15;
            Powr_value += START_POWER;
      }
```

ADC 数据采用右对齐（0-2048），所对应 Powr_value，S16_MAX 最大值为 32767。

4.4.2 占空比处理函数

```
speed_ref_ramp();
```

u16 power_DEC_STEP = 30 //降速步长，power_UP_STEP = 200; //加速步长

速度输入为 0 进入刹车状态：

```
if((t_power == 0) && (mBLDC_CtrProc.eSysState == RUN) && (BLDC_SensorlessCtr.onTheFlyFlg == 0))
BLDC_SensorlessCtr.EabsFlg = 1;                    //Eabs 标志位置 1
mBLDC_CtrProc.eSysState = BRAKE;                   //跳转至刹车状态机
```

短下桥：

```
MCPWM_IO01 = DRIVER_POLARITY;
MCPWM_IO23 = DRIVER_POLARITY;
```

即低速状态（ADC 模式）下，占空比限制最大输出为 75%：

```
t_value = sat(t_value, PWM_PERIOD * 0.1, PWM_PERIOD * 0.75)
```

若调试过程中电机转速达不到最高，且占空比没有达到 100%，说明没有切换到 CMP 模式。

程序中对启动状态下前几拍进行限制，以提高起动转矩：

```
t_value = sat(t_value, PWM_PERIOD * 0.15, PWM_PERIOD * 0.75);
```

占空比转换过程参数传递关系如下：ADC 采样值

-->Speed_value（*转换系数）

--> Powr_value

--> mBLDC_CtrProc.nPower

--> t_power（±加速、减速步长）

--> mBLDC_CtrProc.nRequestPower

--> t_value = (((s32)(mBLDC_CtrProc.nRequestPower) * PWM_PERIOD) >> 15)

--> mBLDC_CtrProc.nBLDC_PWM_value（占空比寄存器值）

4.4.3 延时断电函数

```
Dely_Turn_Off();
```

扳机松开后，延迟一段时间后再进行断电，期间需要继续给 LED 供电，在“hardware_config.h”中设置。

```
#define LED_DELAY_TIME                  10000          /* 松扳机后延时关机时间 */
```

4.4.4 LED 显示函数

```
LED_display();
```

开关关闭状态时 LED 常亮，若开关闭合时有故障，LED 闪烁报警，根据不同故障指示，LED 闪烁不不同。

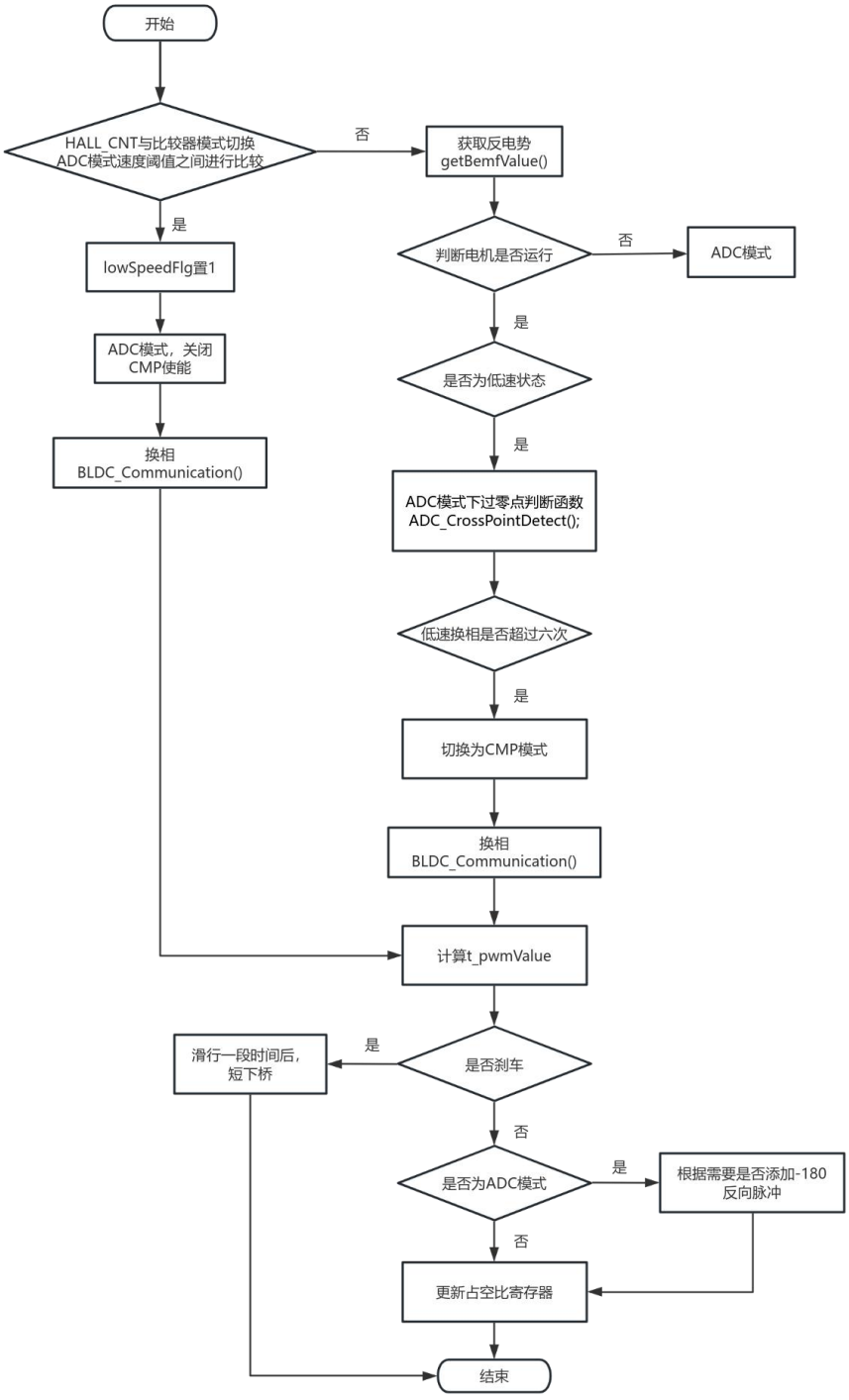
例如：当为过压故障时，LED 闪烁三次。

```
if(mBLDC_CtrProc.sys_error_flg & HIG_VOL_ERROR)
{
LED_flicker_flg = 3;
}
```

4.5 中断程序处理

4.5.1 ADC 中断函数

void BLDC_CommProcess(void)



说明：

HALL_CNT 用来计算 30，60 度时间，值越大，说明电机运行速度越慢。

BLDC_SensorlessCtr.fir_speed 为速度参考变量，是对应的时间（60 度的），时间越短，对应的转速就越快，不是真实速度。

判断 if(BLDC_SensorlessCtr.fir_speed <= WORK_MODE_THRESHLD)

WORK_MODE_THRESHLD 为 ADC 模式切换比较器模式速度阈值，若想让系统一直工作在 ADC 模式，可将此值设置为较小的数值，在“MC_Parameter.h”中设置。

bus_current_limPro(); 限流保护函数

大于设定最大值时，降低占空比，控制电流大小，控制电流值恒定。

t_pwmValue 为最终给到占空比寄存器的值。

ADC 采样点设置：避开 mos 震荡区域，平稳之后，PWM 结束之前采样。

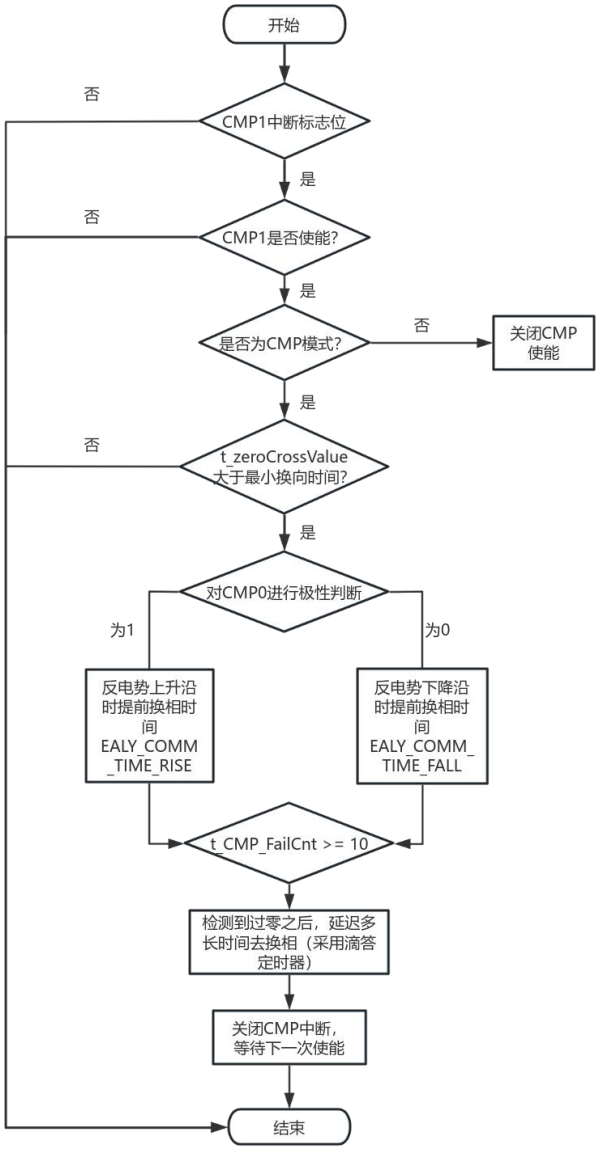
```
if(t_pwmValue > 300){
    MCPWM_TMR0 =   t_pwmValue - 200;
}
else{
    MCPWM_TMR0 = 50;
}
```

4.5.2 CMP 中断函数

void BLDC_CMP_ModeControl(void)

CMP1 为硬件过流，与 MCPWM 模块联动触发，不要需要进行中断服务函数处理。

CMP0 主要应用于比较器模式下过零点检测：



说明：

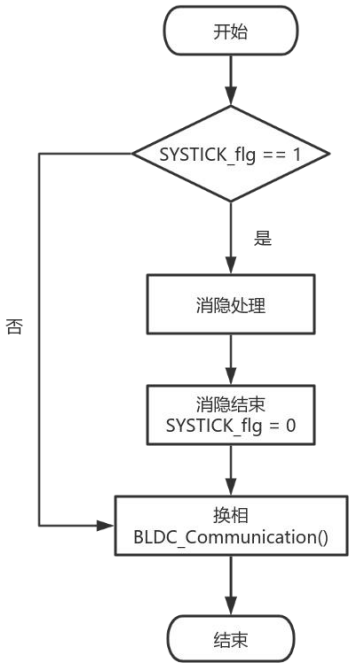
EALY_COMM_TIME_RISE，EALY_COMM_TIME_FALL 为分别为反电势上升沿，下降沿时提前换相时间。

t_CMP_FailCnt >= 10 为软件滤波处理，防止误判。

4.5.3 SysTick_Handler 中断函数

BLDC_DelayCommunication();

系统滴答定时中断：



滴答定时中断主要用于消隐处理，过零检测之后，当 30 度延迟换相时间已经达到，进行换相操作 BLDC_Communication();

第五章 综合调试

5.1 硬件通路配置

在“hardware_config.h”中，根据硬件原理图修改所对应的 ADC 通道，以及 CMP 端口设置，具体原理详见 3.1 章节中说明。

```
/* ------------------------------ADC通道号定义------------------------------ */
#define ADC_CHANNEL_OPA0              ADC_CHANNEL_0
#define ADC_CHANNEL_OPA1              ADC_CHANNEL_8

/* ADC相电流采样时序，硬件相关 ------------------------------------------------ */
/* Porting Application Notice 注意采样序列 ----------------------------------- */
#define ADC_BUS_CUR_CHANNEL           ADC_CHANNEL_OPA0     /* 母线电流 */
#define ADC_PEAK_CUR_CHANNEL          ADC_CHANNEL_OPA0

#define ADC_BUS_VOL_CHANNEL           ADC_CHANNEL_10       /* 母线电压 */
#define ADC_SPEED_CHANNEL             ADC_CHANNEL_5        /* 速度信号 */

#define ADC_MOS_TEMP_CHANNEL          ADC_CHANNEL_3        /* MOS温度 */
#define ADC_BAT_TEMP_CHANNEL          ADC_CHANNEL_10       /* 电池温度 */

#define ADC_OPA_OUT_CHANNEL           ADC_CHANNEL_5       /* 运放输出采样 */

#define BEMF_CH_A                     ADC_CHANNEL_1        /* ADC_Ch1 */
#define BEMF_CH_B                     ADC_CHANNEL_2        /* ADC_Ch2 */
#define BEMF_CH_C                     ADC_CHANNEL_4        /* ADC_Ch3 */
```

```
/* ---------------------------CMP比较器设置相关定义 --------------------------- */
#define CMP1_SELN_SET                 CMP_SELN_DAC            /* 比较器1负端选择DAC */
#define CMP1_SELP_SET                 CMP_SELP_OPA0_OUT       /* 比较器1正端选择运放1输出结果 */

#define CMP0_SELN_SET                 CMP_SELN_HALL_MID       /* 比较器0负端选择HALL MID */
//#define CMP0_SELP_SET                 CMP0_SELP_CMP0_IP1       /* 比较器0正端选择IP0 */
#define CMP_IP_A                      CMP_SELP_IP2          /* A相反电势电压比较器通道 */
#define CMP_IP_B                      CMP_SELP_IP1          /* B相反电势电压比较器通道 */
#define CMP_IP_C                      CMP_SELP_IP3          /* C相反电势电压比较器通道 */
```

5.2 硬件通路测试

在“fuction.h”当中，选择 DEBUG_PWM_OUTPUT 为 TEST_ON，设置电源限流 0.1A 左右，测试三相对地的波形。注意示波器的地线不要接，以免误操作，烧坏示波器。

```
#define DEBUG_PWM_OUTPUT              TEST_ON       /* PWM 在调试的时候输出一定占空比 */
```

对应代码：

```
#if (DEBUG_PWM_OUTPUT == 0)
DebugPWM_OutputFunction();                          /* 调试的时候输出 25%的 PWM 波形 */
#endif
```

如果输出不正确，要检查时程序配置的问题，还是硬件电路的问题，例如 MOS 管或者驱动芯片损坏，测试信号正常后才能进入下一步调试。

5.3 硬件参数配置

在“MC_Parameter.h”当中，根据实际应用以及硬件电路设计，配置 ADC 基准电压，计算出运放放大倍数，采样电阻阻值，母线以及反电势分压比。

```
/* ----------------------------Hardware Parameter---------------------------- */
#define ADC_SUPPLY_VOLTAGE            (3.6)              //单位：V  ADC基准电压，3.6或者2.4，大部分应用选择3.6
#define AMPLIFICATION_GAIN            (18.18)          //运放放大倍数
#define RSHUNT                         (0.001)            //单位：Ω  采样电阻阻值
#define VOLTAGE_SHUNT_RATIO           (1.0/(20.0+1.0)) //母线电压分压比
#define BEMF_SHUNT_RATIO              (1.0/(20.0+1.0)) //反电势电压分压比
```

5.4 限流以及短路保护参数配置

最大限流电流 MAX_BUS_CURRENT_SETTINT，在 ADC 中断服务函数 BLDC_CommProcess()中的 bus_current_limPro()当中使用，大于设定最大值时，降低占空比，控制电流大小，控制电流值恒定。

短路电流 SHORT_BUS_CURRENT，为硬件过流设置值，当超过此值时，产生 Fail 事件，关闭 MCPWM 输出，详见第三章节 CMP 和 DAC 模块说明。

```
/* ----------------------------Current Lim Parameter---------------------------- */
#define MAX_BUS_CURRENT_SETTINT       (u16)20 /* 最大限流电流 */
#define CURRENT_ADC_PER_A             (RSHUNT * AMPLIFICATION_GAIN * 32752/3.6) /* 每安电流ADC值 */
#define CURRENT_LIM_VALUE             (u16)(MAX_BUS_CURRENT_SETTINT * CURRENT_ADC_PER_A) /* 最大电流ADC值 */

/* ----------------------------Current Lim SHORT---------------------------- */

#define SHORT_BUS_CURRENT             (u16)20 /* 短路电流 */
#define SHORT_CURRENT_VOL             (SHORT_BUS_CURRENT * RSHUNT * AMPLIFICATION_GAIN *10000)
#define SHORT_CURRENT_DAC             (u16)((SHORT_BUS_CURRENT * RSHUNT * 256)/3 )     /* 短路电压对应DAC值 */
```

5.5 分极限流参数配置

对应 OVER_current_protect()函数，详见 4.3 章节说明。

```
/* ----------------------------Current Lim Parameter---------------------------- */
#define BUS_CURRENT_FIRST             (u16)30 /* 一级限流 */
#define BUS_CURRENT_SECOND            (u16)50 /* 二级限流 */

#define CURRENT_ADC_PER_A             (RSHUNT * AMPLIFICATION_GAIN * 32752/3.6) /* 每安电流ADC值 */

#define OVER_CURRENT_FIRST_THD        (u16)(BUS_CURRENT_FIRST * CURRENT_ADC_PER_A)  /* 第一级限流电流ADC值 */
#define OVER_CURRENT_SECOND_THD       (u16)(BUS_CURRENT_SECOND * CURRENT_ADC_PER_A) /* 第二级限流电流ADC值 */

#define TIME_LIMIT_FIRST              1000  /* 一级限流时间 */
#define TIME_LIMIT_SECOND             200   /* 二级限流时间 */
```

5.6 电压保护参数配置

对应 Vol_protect()函数，详见 4.3 章节说明。

```
/* ----------------------------Voltage Protect Parameter---------------------------- */
#define LOW_VOLATAGE_THD              7    /* 欠压电压 */
#define RSM_LO_VOLATAGE_THD           7.5 /* 欠压恢复电压 */
#define OV_VOLTAGE_THD                25    /* 过压门槛 */
#define RSM_OV_VOLTAGE_THD            24    /* 过压恢复门槛 */

#define LOW_VOLATAGE_THD_1            5     /* 第一段欠压，  */
#define LOW_VOLATAGE_THD_2            5.5   /* 欠压电压 */

#define FIRST_LOW_TIME                500    /* 第一段欠压时间 */
```

5.7 初始位置检测参数配置

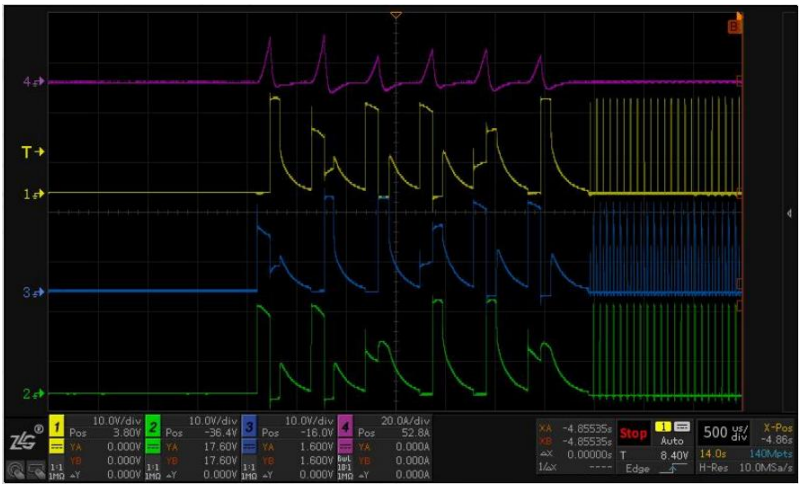
通过调整脉宽注入的时间宽度和空闲等待时间的大小，去判断检测的精度，一般来说，脉冲注入时间宽度越大，精度越高，同时带来的噪声也越大。

```
/* ----------------------IPD 转子初始位置检测 脉冲注入时间设置---------------- */
#define SEEK_POSITION_STATUS            (TRUE)              //初始位置检测状态 TRUE为使能, FALSE为不使能
#define U_START_ANGLE_COMP              (0)                 //单位:度 初始位置检测补偿角度

#define IPD_PLUS_TIME_SETTING           (20)               /* 脉冲注入时间宽度设置 单位us */
#define IPD_WAIT_TIME_SETTING           (400)               /* 脉冲空闲等待时间宽度设置 单位us */

#define IPD_PLUS_TIME_WIDTH             (u32)(IPD_PLUS_TIME_SETTING*(MCU_MCLK/1000000))    /* 脉冲注入时间宽度设置 单位clk */
#define IPD_PLUS_WAIT_TIME              (u32)(IPD_WAIT_TIME_SETTING*(MCU_MCLK/1000000))    /* 脉冲空闲等待时间宽度设置 单位clk */
```

下图为基于磁饱和效应的六脉冲定位法进行初始脉冲检测，凸极性高的电机六个脉冲信号错落有致，定位准确率高：



5.8 启动参数配置

STARTUP_HAlL_COMM_TIME，STARTUP_HAlL_COMM_TIME，EXOVERFOLLW_HAlL_COMM_TIME 为换相时间参数，在 CMP 中断的 BLDC_CMP_ModeControl()函数当中使用，一般情况下不需要改动。

WORK_MODE_THRESHLD 为 ADC 模式切换比较器模式速度阈值，若想系统一直工作在 ADC 模式下，可将此值调小。

QUIT_CMP_THRESHLD 为比较器模式切换 ADC 模式速度阈值。

HFI_PWM_WIDTH 为 ADC 模式下反向 180 度脉冲注入脉冲宽度设置，PWM 开关周期开始时注入。

```
/* ------------------------------启动参数设置------------------------------------ */
#define STARTUP_HAlL_COMM_TIME          10000            /* 启动时，最小换相时间 单位：HALL模块时钟周期数 */
#define STARTUP_HAlL_COMM_TIMEEX        500000          /* 比较器工作模式下最大换相时间 单位：HALL模块时钟周期数 */
#define OVERFOLLW_HAlL_COMM_TIME        260000           /* 超时换相时间设置 单位：HALL模块时钟周期数 */

#define WORK_MODE_THRESHLD              150000//90000          /* ADC模式切换比较器模式速度阈值 */
#define QUIT_CMP_THRESHLD               210000                /* 比较器模式切换ADC模式速度阈值 */

#define HFI_PWM_WIDTH                   100                /* 脉冲注入宽度 */
#define HFI_PWM_MIN                     ((s16)(-PWM_PERIOD + 2))
#define HFI_PWM_PLUS                    ((s16)(-PWM_PERIOD + HFI_PWM_WIDTH))
```

5.9 换相参数配置

重叠换相时间 RE_COMM_CNT 在 BLDC_Communication() 函数中使用，主要用于消隐处理。

提前换相时间 EALY_COMM_TIME_RISE，EALY_COMM_TIME_FALL 分别为反电势上升沿，下降沿时提前换相时间，将此时间值调小，可以让电机达到提前换相的效果，从而增大转矩，但会增加电机运行时的波动。

```
/* ------------------------------重叠换相时间设置----------------------------- */
#define RE_COMM_TIME                    1                /* 重叠换相时间 单位：us */
#define RE_COMM_CNT                     (RE_COMM_TIME*MCU_MCLK/1000000) /* 标幺化的重叠换相时间 单位：Clk */

/* ------------------------ BLDC SensorLess提前换相时间 ---------------------- */
#define EALY_COMM_TIME_RISE             (s16)(-2000)        /* 反电势上升沿时提前换相时间 单位：HALL模块时钟周期数 */
#define EALY_COMM_TIME_FALL             (s16)(-2000)        /* 反电势下降沿时提前换相时间 单位：HALL模块时钟周期数 */
```

5.10 顺风启动参数配置

ON_FLY_RUN_TIME 为顺风检测时间。

ON_FLY_MAX_DUTY，ON_FLY_MIN_DUTY 为顺风启动时占空比范围值。

ON_FLY_VOL 为顺风启动门槛，大于此值顺风启动。

```
/* ---------------------------------- 顺风启动功能 ------------------------------ */
#define ON_FLY_START_FUNCTION           DISABLE_FUNCTION        /* 顺风起功能 */

#define ON_FLY_RUN_TIME                 (u16)20  //顺风时间设置，单位：ms

#define ON_FLY_MAX_DUTY                 (u16)(0.8 * PWM_PERIOD)
#define ON_FLY_MIN_DUTY                 (u16)(0.2 * PWM_PERIOD)

#define MOTOR_MIN_SPEED                 (u32)37500  //最低电压下最小运行转速
#define PER_ONE_DUTY_ADD_SPEED          (u32)1180   //每升高1%占空比电机提升的转速

#define ON_FLY_MAX_POWER                (u16)(ON_FLY_MAX_DUTY * 32767/PWM_PERIOD)
#define ON_FLY_MIN_POWER                (u16)(ON_FLY_MIN_DUTY * 32767/PWM_PERIOD)

#define ON_FLY_VOL                      (u16)1    //(顺风起判断门槛，单位：V)
#define ON_FLY_ADC_THD                  (u16)(ON_FLY_VOL * BEMF_SHUNT_RATIO*32767/ADC_SUPPLY_VOLTAGE)
```

5.11 Eabs 刹车功能参数配置

若 EABS_HARD = ENABLE_FUNCTION，则 BLDC_SensorlessCtr.Eabs_Hard_Flg 置 1，为硬刹车模式，刹车时进行短下桥处理。

若 EABS_HARD = DISABLE_FUNCTION，则为软刹车模式，根据升压斩波原理，可通过合理控制各桥臂功率管的通断，实现回馈充电，软刹模式升压较大不推荐用。

EABS_SLIP_TIME 为刹车前滑行时间。

```
/* ---------------------------------- EABS功能相关设置 ---------------------------------- */
#define EABS_FUNCTION                 ENABLE_FUNCTION         /* Eabs刹车功能 */

#define EABS_HARD                      ENABLE_FUNCTION        /* 使能后刹车短三相下管，否则为软刹车，三相下管以一定占空比输出短接 */
                                                               /* PS:在用稳压电源调试软刹车时，电源可能会产生负电流保护，用电池包会好一些 */
                                                               /* PS:建议使用硬刹 */
#define EABS_SLIP_TIME                (u16) 5                    /* 刹车前滑行时间设置 */
#define EABS_RUN_TIME                 (u16) 200                  /* 刹车时间设置 */
#define EABS_TOTAL_TIME               (u16)(EABS_SLIP_TIME + EABS_RUN_TIME)
#define SOFT_EABS_STEP                (u16)50                    /* 软刹车加速步长，占空比一直加大直到开满 */
#define SOFT_EABS_MIN_DUTY            (u16) 1000              /* 软刹车基础占空比 */
```

5.12 温度保护参数配置

可根据实际产品设计需求，对 MOS 温度以及 BAT 温度保护进行配置，具体实现逻辑在 TEMP_protect()函数中。

```
/**************************MOS温度保护************************************/
#define MOS_TEMP_UP_VOL               5     //MOS温度检测上拉电压，单位：V
#define MOS_TEMP_UP_RES               10    //MOS温度检测上拉电阻，单位：KΩ

#define MOS_TEMP_OVER_TIME            500

#define MOS_TEMP_OVER_RES             1.11  //MOS过温时NTC阻值，80℃对应1.67K  6512
#define RSM_MOS_TEMP_OVER_RES         1.2   //MOS过温恢复NTC阻值，75℃对应1.98K  7520

#define MOS_TEMP_OVER_THD             (u32)((MOS_TEMP_OVER_RES * MOS_TEMP_UP_VOL * 32767)/((MOS_TEMP_OVER_RES + MOS_TEMP_UP_RES) * 3.6))
#define RSM_MOS_TEMP_OVER_THD         (u32)((RSM_MOS_TEMP_OVER_RES * MOS_TEMP_UP_VOL * 32767)/((RSM_MOS_TEMP_OVER_RES + MOS_TEMP_UP_RES) * 3.6))

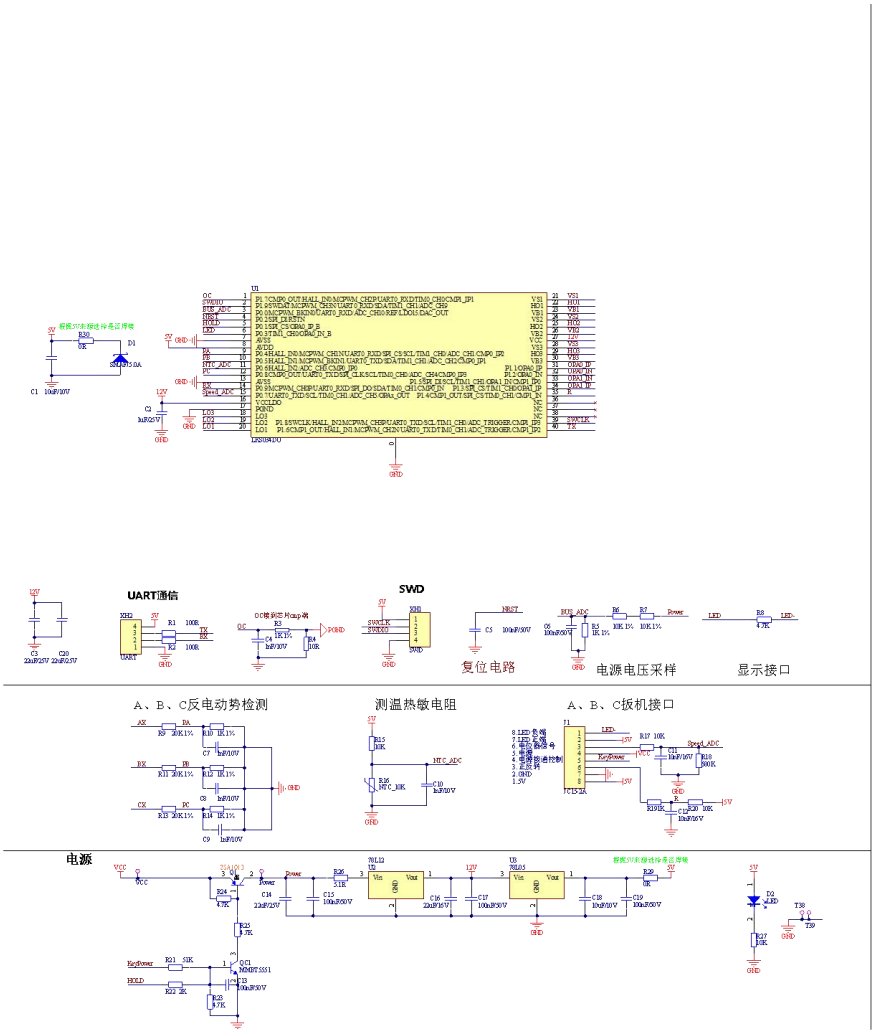
/**************************BAT温度保护************************************/
#define BAT_TEMP_UP_VOL               5     //电池温度检测上拉电压，单位：V
#define BAT_TEMP_UP_RES               10    //MOS温度检测上拉电阻，单位：KΩ

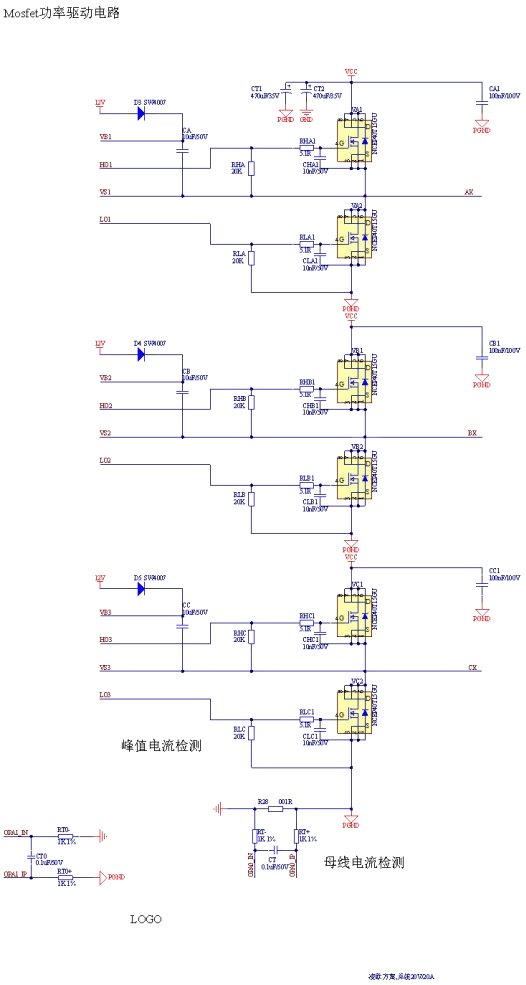
#define BAT_TEMP_OVER_TIME            500

#define BAT_TEMP_OVER_RES             1.11  //电池过温时NTC阻值，80℃对应1.67K  6512
#define RSM_BAT_TEMP_OVER_RES         1.2   //电池过温恢复NTC阻值，75℃对应1.98K  7520

#define BAT_TEMP_OVER_THD             (u32)((BAT_TEMP_OVER_RES * BAT_TEMP_UP_VOL * 32767)/((BAT_TEMP_OVER_RES + BAT_TEMP_UP_RES) * 3.6))
#define RSM_BAT_TEMP_OVER_THD         (u32)((RSM_BAT_TEMP_OVER_RES * BAT_TEMP_UP_VOL * 32767)/((RSM_BAT_TEMP_OVER_RES + BAT_TEMP_UP_RES) * 3.6))
```

附录 1　LKS034DO 电动工具类 DEMO 板原理图





附录 2　MC_Parameter.h 参数说明

```
/* ---------------------------------功能相关定义---------------------------- */
#define    ENABLE_FUNCTION                1
#define    DISABLE_FUNCTION               0
/* ---------------------------------  顺风启动功能  ---------------------------- */
#define ON_FLY_START_FUNCTION          DISABLE_FUNCTION        /*  顺风起功能  */

#define ON_FLY_RUN_TIME                (u16)20   //顺风时间设置，单位：ms
```

```
#define ON_FLY_MAX_DUTY                    (u16)(0.8 * PWM_PERIOD)
#define ON_FLY_MIN_DUTY                    (u16)(0.2 * PWM_PERIOD)

#define MOTOR_MIN_SPEED                    (u32)37500   //最低电压下最小运行转速
#define PER_ONE_DUTY_ADD_SPEED             (u32)1180    //每升高 1%占空比电机提升的转速

#define ON_FLY_MAX_POWER                   (u16)(ON_FLY_MAX_DUTY * 32767/PWM_PERIOD)
#define ON_FLY_MIN_POWER                   (u16)(ON_FLY_MIN_DUTY * 32767/PWM_PERIOD)

#define ON_FLY_VOL                         (u16)1       //(顺风起判断门槛，单位：V)
#define    ON_FLY_ADC_THD                                                    (u16)(ON_FLY_VOL    *
BEMF_SHUNT_RATIO*32767/ADC_SUPPLY_VOLTAGE)

/* ------------------------------- EABS 功能相关设置  ------------------------------ */
#define EABS_FUNCTION                      ENABLE_FUNCTION            /* Eabs 刹车功能  */
#define EABS_HARD                          ENABLE_FUNCTION
 /*  使能后刹车短三相下管，否则为软刹车，三相下管以一定占空比输出短接  */
/* PS:在用稳压电源调试软刹车时，电源可能会产生负电流保护，用电池包会好一些  */
/* PS:建议使用硬刹  */

#define EABS_SLIP_TIME                     (u16) 5                    /*  刹车前滑行时间设置  */
#define EABS_RUN_TIME                      (u16) 200                  /*  刹车时间设置  */
#define EABS_TOTAL_TIME                    (u16)(EABS_SLIP_TIME + EABS_RUN_TIME)
#define SOFT_EABS_STEP                     (u16)50
 /*  软刹车加速步长，占空比一直加大直到开满  */
#define SOFT_EABS_MIN_DUTY                 (u16) 1000                 /*  软刹车基础占空比  */

/* ------------------------------Hardware Parameter---------------------------- */
#define ADC_SUPPLY_VOLTAGE                 (3.6)
//单位: V    ADC 基准电压，3.6 或者 2.4,大部分应用选择 3.6
#define AMPLIFICATION_GAIN                 (18.18)            //运放放大倍数
#define RSHUNT                             (0.001)              //单位:  Ω    采样电阻阻值
#define VOLTAGE_SHUNT_RATIO                (1.0/(20.0+1.0)) //母线电压分压比
#define BEMF_SHUNT_RATIO                   (1.0/(20.0+1.0)) //反电势电压分压比
```

```
/* ----------------------------Current Lim Parameter-------------------------- */
#define MAX_BUS_CURRENT_SETTINT            (u16)20        /* 最大限流电流 */
#define CURRENT_ADC_PER_A                  (RSHUNT * AMPLIFICATION_GAIN * 32752/3.6)
 /* 每安电流 ADC 值 */
#define  CURRENT_LIM_VALUE                                 (u16)(MAX_BUS_CURRENT_SETTINT  *
CURRENT_ADC_PER_A) /* 最大电流 ADC 值 */

/* ----------------------------Current Lim SHORT------------------------- */
#define SHORT_BUS_CURRENT                  (u16)20 / * 短路电流 */
#define  SHORT_CURRENT_VOL                                  (SHORT_BUS_CURRENT * RSHUNT *
AMPLIFICATION_GAIN *10000)
#define SHORT_CURRENT_DAC                  (u16)((SHORT_BUS_CURRENT * RSHUNT * 256)/3 )
/* 短路电压对应 DAC 值 */
#define SHORT_PROTECT_DAC                  (u16)(OPA_OUT_BASE_DAC + SHORT_CURRENT_DAC)
 /* 短路保护对应的 DAC 值 = 运放基础输出 + 设定的短路保护锁对应的 DAC */

/* ----------------------------Current Lim Parameter-------------------------- */
#define BUS_CURRENT_FIRST                  (u16)30 /* 一级限流 */
#define BUS_CURRENT_SECOND                 (u16)50 /* 二级限流 */

#define CURRENT_ADC_PER_A                  (RSHUNT * AMPLIFICATION_GAIN * 32752/3.6)
 /* 每安电流 ADC 值 */
#define OVER_CURRENT_FIRST_THD             (u16)(BUS_CURRENT_FIRST * CURRENT_ADC_PER_A)
/* 第一级限流电流 ADC 值 */
#define OVER_CURRENT_SECOND_THD      (u16)(BUS_CURRENT_SECOND * CURRENT_ADC_PER_A)
/* 第二级限流电流 ADC 值 */

#define TIME_LIMIT_FIRST                   1000   /* 一级限流时间 */
#define TIME_LIMIT_SECOND                  200    /* 二级限流时间 */

/* ----------------------------Voltage Protect Parameter-------------------------- */
#define LOW_VOLATAGE_THD                     7    /* 欠压电压 */
#define RSM_LO_VOLATAGE_THD                  7.5 /* 欠压恢复电压 */
#define OV_VOLTAGE_THD                       25    /* 过压门槛 */
```

```
#define RSM_OV_VOLTAGE_THD                  24    /* 过压恢复门槛 */

#define LOW_VOLATAGE_THD_1                  5     /* 第一段欠压， */
#define LOW_VOLATAGE_THD_2                  5.5   /* 欠压电压 */

#define FIRST_LOW_TIME                      500   /* 第一段欠压时间 */

#define  LOW_VOLATAGE_FIRST                                  (u16)(LOW_VOLATAGE_THD_1 *
VOLTAGE_SHUNT_RATIO/3.6*32768)
#define  LOW_VOLATAGE_SECOND                                 (u16)(LOW_VOLATAGE_THD_2 *
VOLTAGE_SHUNT_RATIO/3.6*32768)

#define  LOW_VOLATAGE_ADC                                    (u16)(LOW_VOLATAGE_THD *
VOLTAGE_SHUNT_RATIO/3.6*32768)
#define  RSM_LO_VOLATAGE_ADC                                 (u16)(RSM_LO_VOLATAGE_THD *
VOLTAGE_SHUNT_RATIO/3.6*32768)
#define  OV_VOLTAGE_ADC                                      (u16)(OV_VOLTAGE_THD *
VOLTAGE_SHUNT_RATIO/3.6*32768)
#define  RSM_OV_VOLTAGE_ADC                                  (u16)(RSM_OV_VOLTAGE_THD *
VOLTAGE_SHUNT_RATIO/3.6*32768)

/* ----------------------IPD 转子初始位置检测 脉冲注入时间设置---------------- */
#define SEEK_POSITION_STATUS                (TRUE)              //初始位置检测状态 TRUE 为使能,
FALSE 为不使能
#define U_START_ANGLE_COMP                  (0)                 //单位:度 初始位置检测补偿角度

#define IPD_PLUS_TIME_SETTING               (20)                /* 脉冲注入时间宽度设置 单位 us */
#define IPD_WAIT_TIME_SETTING               (400)               /* 脉冲空闲等待时间宽度设置 单位 us */

#define IPD_PLUS_TIME_WIDTH                 (u32)(IPD_PLUS_TIME_SETTING*(MCU_MCLK/1000000))
/* 脉冲注入时间宽度设置 单位 clk */
#define IPD_PLUS_WAIT_TIME                  (u32)(IPD_WAIT_TIME_SETTING*(MCU_MCLK/1000000))
/* 脉冲空闲等待时间宽度设置 单位 clk */
```

```
/* ----------------------------启动参数设置---------------------------------- */
#define STARTUP_HAIL_COMM_TIME              10000               /* 启动时，最小换相时间 单位：HALL 模块时钟周期数 */
#define STARTUP_HAIL_COMM_TIMEEX            500000              /* 比较器工作模式下最大换相时间 单位：HALL 模块时钟周期数 */
#define OVERFOLLW_HAIL_COMM_TIME            260000              /* 超时换相时间设置 单位：HALL 模块时钟周期数 */

#define WORK_MODE_THRESHLD                  150000//90000     /* ADC 模式切换比较器模式速度阈值 */
#define QUIT_CMP_THRESHLD                   210000            /*比较器模式切换 ADC 模式速度阈值 */

#define HFI_PWM_WIDTH                       100                 /* 脉冲注入宽度 */
#define HFI_PWM_MIN                         ((s16)(-PWM_PERIOD + 2))
#define HFI_PWM_PLUS                        ((s16)(-PWM_PERIOD + HFI_PWM_WIDTH))

/* ------------------------------重叠换相时间设置---------------------------- */
#define RE_COMM_TIME                        1                   /* 重叠换相时间 单位：us */
#define RE_COMM_CNT                         (RE_COMM_TIME*MCU_MCLK/1000000)
/* 标幺化的重叠换相时间 单位：Clk */

/* ------------------------ BLDC SensorLess 提前换相时间 ---------------------- */
#define EALY_COMM_TIME_RISE                 (s16)(-2000)        /* 反电势上升沿时提前换相时间 单位：HALL 模块时钟周期数 */
#define EALY_COMM_TIME_FALL                 (s16)(-2000)        /* 反电势下降沿时提前换相时间 单位：HALL 模块时钟周期数 */

/* ----------------------------direction check Parameter---------------------- */
#define CW                                  (0)                 /* 电机转向：顺时针 */
#define CCW                                 (1)                 /* 电机转向：逆时针*/

/* ----------------------------ADC 校准相关参数设置---------------------------- */
#define CALIB_SAMPLES                       (512)               //ADC 偏置校准次数，不可修改
#define OFFSET_THD                          (3500)              //ADC 偏置误差阈值，不用修改
```

```
/* ----------------------------预驱自举电容预充电参数---------------------------- */
#define CHARGE_TIME                         (100)           //每相预充电时间，根据实际硬件参数修改

/*-----------------------------------扳机输入相关参数-------------------------*/
#define START_THRESHOLD_ADC       300
#define START_POWER                     2000

/**************************MOS 温度保护************************************/
#define MOS_TEMP_UP_VOL                 5       //MOS 温度检测上拉电压，单位：V
#define MOS_TEMP_UP_RES                 10      //MOS 温度检测上拉电阻，单位：KΩ
#define MOS_TEMP_OVER_TIME              500

#define MOS_TEMP_OVER_RES               1.11    //MOS 过温时 NTC 阻值，80℃对应 1.67K   6512
#define RSM_MOS_TEMP_OVER_RES           1.2     //MOS 过温恢复 NTC 阻值，75℃对应 1.98K   7520

#define MOS_TEMP_OVER_THD               (u32)((MOS_TEMP_OVER_RES * MOS_TEMP_UP_VOL * 32767)/((MOS_TEMP_OVER_RES + MOS_TEMP_UP_RES) * 3.6))
#define   RSM_MOS_TEMP_OVER_THD         (u32)((RSM_MOS_TEMP_OVER_RES   * MOS_TEMP_UP_VOL * 32767)/((RSM_MOS_TEMP_OVER_RES + MOS_TEMP_UP_RES) * 3.6))

/**************************BAT 温度保护************************************/
#define BAT_TEMP_UP_VOL                 5       //电池温度检测上拉电压，单位：V
#define BAT_TEMP_UP_RES                 10      //MOS 温度检测上拉电阻，单位：KΩ

#define BAT_TEMP_OVER_TIME              500

#define BAT_TEMP_OVER_RES               1.11    //电池过温时 NTC 阻值，80℃对应 1.67K   6512
#define RSM_BAT_TEMP_OVER_RES           1.2     //电池过温恢复 NTC 阻值，75℃对应 1.98K   7520

#define BAT_TEMP_OVER_THD               (u32)((BAT_TEMP_OVER_RES * BAT_TEMP_UP_VOL * 32767)/((BAT_TEMP_OVER_RES + BAT_TEMP_UP_RES) * 3.6))
#define   RSM_BAT_TEMP_OVER_THD         (u32)((RSM_BAT_TEMP_OVER_RES   * BAT_TEMP_UP_VOL * 32767)/((RSM_BAT_TEMP_OVER_RES + BAT_TEMP_UP_RES) * 3.6))
```